[인공지능]

Backpropagation

Eunsong Kang
eskang@kangwon.ac.kr

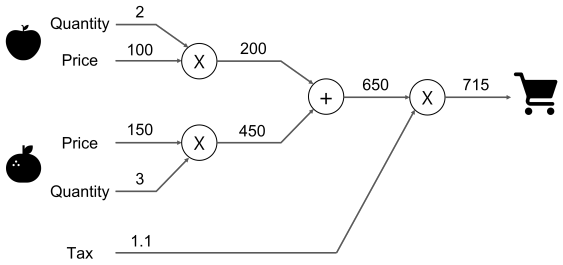
2026. 11. 12.

참고 자료

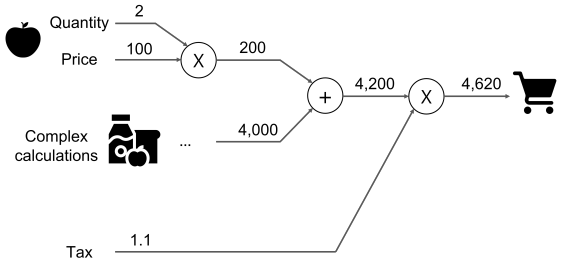
- 경사하강법: AI가 궁금하다면 봐야할 기초영상 경사하강법 - DL2
- 오차역전파법(개념): 백프로파게이션 (BackPropagation) 개념적으로 이해하기 (쉬움) - DL3
- 오차역전파법(수식): 백프로파게이션 두번째 영상! 실제 계산은 어떻게 하는지 살펴보기 - DL4

Computational Graph

- A **computation graph** represents a computational process as a graph, where nodes and edges express the relationships.



- The key feature of a computation graph is that it obtains the final result by propagating local computations.
  - Local means "within a small scope directly related to itself."
  - Each node produces an output only based on the information directly connected to it, regardless of what happens in the rest of the graph.

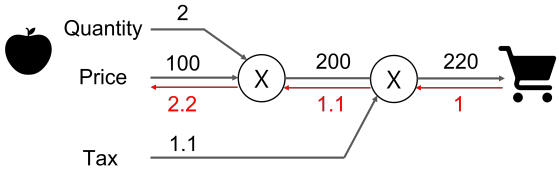


Example: Calculating the total cost when purchasing several items.

- A computation graph allows efficient computation of derivatives through **backpropagation**.
- Examples

  How does the total payment change as the price of apples changes?

→ The derivative of the payment amount (L) with respect to the apple price (x), $\frac{\partial L}{\partial x}$.



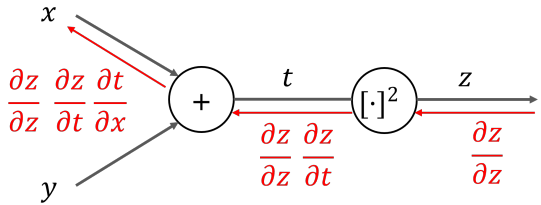
Example: the propagation of derivative values through backpropagation

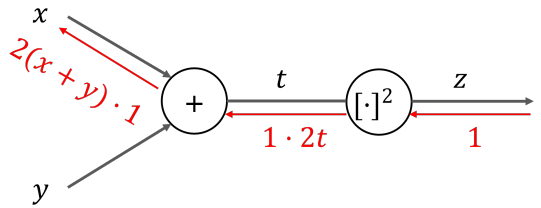
Chain Rule

- The **chain rule** describes the property of differentiation for a **composite function** (a function composed of multiple functions).
- The derivative of a composite function can be expressed as the product of the derivatives of the functions that compose it.
- Example

$$z = t^2$$
$$t = x + y \tag{1}$$
$$\frac{\partial z}{\partial x} = \frac{\partial z}{\partial t}\frac{\partial t}{\partial x} \tag{2}$$
$$\frac{\partial z}{\partial t} = 2t, \quad \frac{\partial t}{\partial x} = 1 \tag{3}$$
$$\frac{\partial z}{\partial x} = \frac{\partial z}{\partial t}\frac{\partial t}{\partial x} = 2t \cdot 1 = 2(x + y) \tag{4}$$

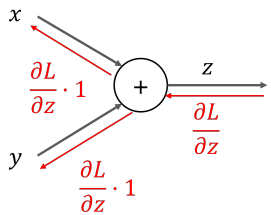
KNU KANGWON NATIONAL UNIVERSITY


x
y
+
t
$[\cdot]^2$
z
$\frac{\partial z}{\partial z}$ $\frac{\partial z}{\partial t}$ $\frac{\partial t}{\partial x}$
$\frac{\partial z}{\partial z}$ $\frac{\partial z}{\partial t}$
$\frac{\partial z}{\partial z}$

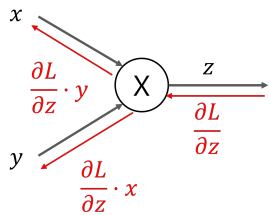

x
y
+
t
$[\cdot]^2$
z
$2(x+y)\cdot 1$
$1 \cdot 2t$
1

Backward Propagation: Addition & Multiplication

- Addition Node (+)

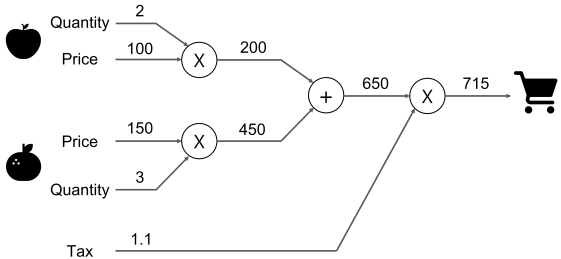


- Multiplication Node (x)



[Exercise]

- Compute the gradients via backpropagation.

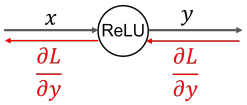


Backward Propagation: ReLU Activation Function

- Definition of ReLU

$$y = \begin{cases} 0, & x \leq 0 \\ x, & x > 0 \end{cases}$$

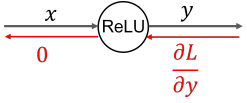
$x > 0$



- The derivative of y with respect to x

$$\frac{\partial y}{\partial x} = \begin{cases} 0, & x \leq 0 \\ 1, & x > 0 \end{cases}$$

$x \leq 0$

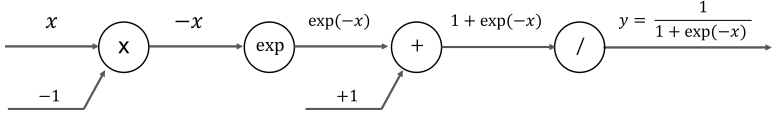


Backward Propagation: Sigmoid Activation Function

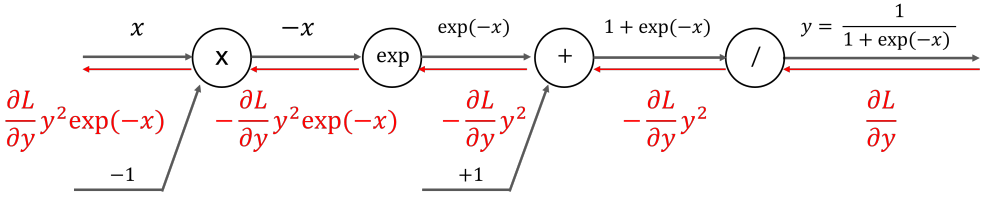
- Definition of Sigmoid function

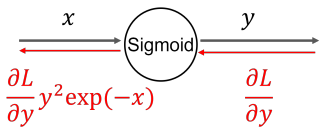
$$y = \frac{1}{1 + \exp(-x)}$$

- The computational graph of sigmoid function



- Backpropagation computation graph of the sigmoid layer



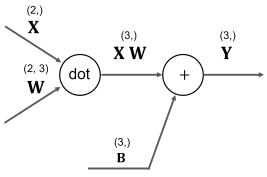


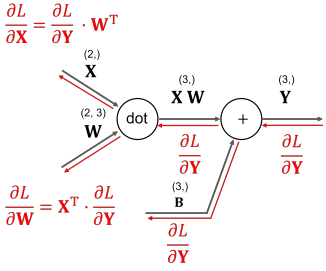
$$\begin{aligned}
\frac{\partial L}{\partial y} y^2 \exp(-x) &= \frac{\partial L}{\partial y} \frac{1}{(1+\exp(-x))^2} \exp(-x) \\
&= \frac{\partial L}{\partial y} \frac{1}{1+\exp(-x)} \frac{\exp(-x)}{1+\exp(-x)} \\
&= \frac{\partial L}{\partial y} y(1-y)
\end{aligned}$$

→ The backpropagation of the sigmoid layer can be computed using only the output (y) from the forward propagation.

Backward Propagation: Affine Transformation

- During the forward pass of a neural network, the matrix multiplication operation corresponds to an affine transformation in geometry.
  - Refer to the component that performs this affine transformation as the affine layer.

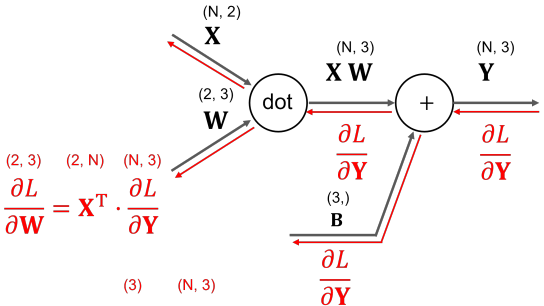




Backward Propagation: Batch Affine Transformation

(N, 2) (N, 3) (3, 2)

$$\frac{\partial L}{\partial \mathbf{X}} = \frac{\partial L}{\partial \mathbf{Y}} \cdot \mathbf{W}^{\mathrm{T}}$$

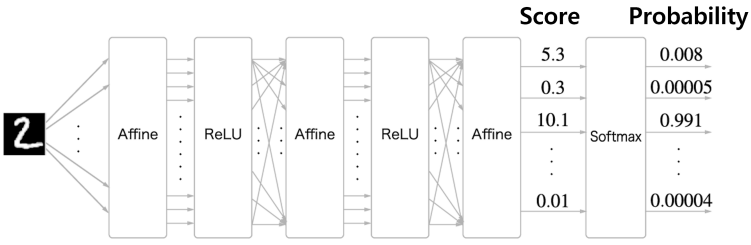


(3) (N, 3)

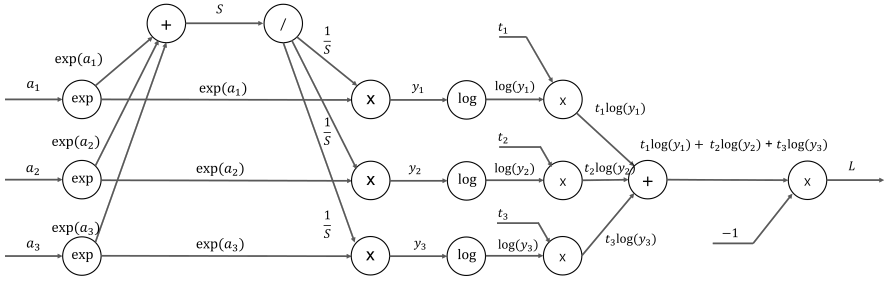
$$\frac{\partial L}{\partial \mathbf{B}} = \frac{\partial L}{\partial \mathbf{Y}}$$ the sum over the first axis (axis 0, column-wise)

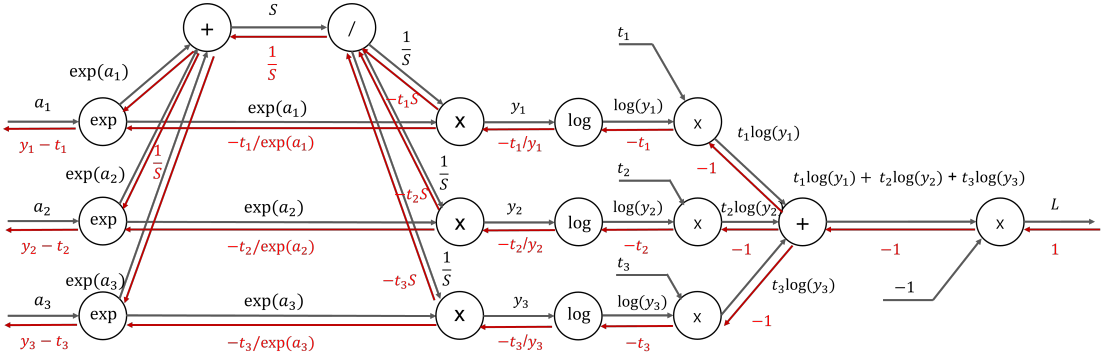
Backward Propagation: Softmax with Loss

- Neural networks with softmax function



- Computational graph of Softmax with cross entropy loss




+
S
/
1/S
1/S
exp(a1)
a1
exp
y1 − t1
exp(a1)
−t1/exp(a1)
−t1S
x
y1
−t1/y1
log
log(y1)
−t1
x
t1log(y1)
t1
exp(a2)
a2
exp
y2 − t2
exp(a2)
−t2/exp(a2)
−t2S
x
y2
−t2/y2
log
log(y2)
−t2
x
t2log(y2)
t2
−1
exp(a3)
a3
exp
y3 − t3
exp(a3)
−t3/exp(a3)
−t3S
x
y3
−t3/y3
log
log(y3)
−t3
x
t3log(y3)
t3
−1
+
t1log(y1) + t2log(y2) + t3log(y3)
−1
x
L
1
−1
Softmax layer
Cross entropy layer

Implementing the Learning Algorithm

- Neural network training typically consists of the following steps:

1. Mini-batch selection
   - Randomly select a small subset of the training data, called a mini-batch.
   - The goal is to minimize the loss function value computed from this mini-batch.
2. Gradient computation
   - Compute the gradient of the loss function with respect to each parameter using the mini-batch.
   - The gradient indicates the direction in which the loss increases most rapidly.
3. Parameter update
   - Update each parameter slightly in the direction opposite to the gradient.
4. Iteration
   - Repeat Steps 1–3 until convergence or for a fixed number of epochs.

Advantages of Backpropagation

- Numerical differentiation is slow but easy to implement.
- Backpropagation is fast but more complex to implement.
- Numerical differentiation is mainly used to verify the correctness of backpropagation.

Thank You!

(Q & A)

References

- 사이토 고키 (2025), 밑바닥부터 시작하는 딥러닝 1(리마스터판), 한빛미디어